

```
import tensorflow as tf
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
import pickle
from tensorflow.keras.utils import image_dataset_from_directory
```

```
training_set = image_dataset_from_directory(
    'train',
    labels="inferred",
    label_mode="categorical",
    class_names=None,
    color_mode="rgb",
    batch_size=32,
    image_size=(224, 224),
    shuffle=True,
    seed=None,
    validation_split=None,
    subset=None,
    interpolation="bilinear",
    follow_links=False,
    crop_to_aspect_ratio=False
)
```

Found 76515 files belonging to 4 classes.

```
validation_set = image_dataset_from_directory(
    'val',
    labels="inferred",
    label_mode="categorical",
    class_names=None,
    color_mode="rgb",
    batch_size=32,
    image_size=(224, 224),
    shuffle=True,
    seed=None,
    validation_split=None,
    subset=None,
    interpolation="bilinear",
    follow_links=False,
    crop_to_aspect_ratio=False
)
```

Found 21861 files belonging to 4 classes.

training_set

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32,
name=None), TensorSpec(shape=(None, 4), dtype=tf.float32, name=None))>
```

INPUT_SHAPE = (224, 224, 3)

```
mobnet = tf.keras.applications.MobileNetV3Large(
    input_shape=INPUT_SHAPE,
    alpha=1.0,
    minimalistic=False,
    include_top=True,
    weights="imagenet",
    input_tensor=None,
    classes=1000,
```

```
pooling=None,
dropout_rate=0.2,
classifier_activation="softmax",
include_preprocessing=True,
# name="MobileNetV3Large",
)
```

```
#mobnet.trainable = False #--used to freeze weights of this pretrained model
```

```
model = tf.keras.models.Sequential()
```

```
model.add(tf.keras.Input(shape=INPUT_SHAPE))
```

```
model.add(mobnet)
```

```
model.add(tf.keras.layers.Dense(units = 4,activation='softmax'))
```

Start coding or [generate](#) with AI.

```
metrics_list = ['accuracy',
                tf.keras.metrics.F1Score()]
```

```
model.compile(optimizer=tf.keras.optimizers.legacy.Adam(
    learning_rate=0.0001),loss='categorical_crossentropy',metrics=metrics_list)
```

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
MobilenetV3large (Function al)	(None, 1000)	5507432
dense (Dense)	(None, 4)	4004

=====
 Total params: 5511436 (21.02 MB)
 Trainable params: 5487036 (20.93 MB)
 Non-trainable params: 24400 (95.31 KB)

```
training_history = model.fit(x=training_set,validation_data=validation_set,epochs=15)
```

```
Epoch 1/15
2392/2392 [=====] - 987s 411ms/step - loss: 1.1454 - accuracy: 0.842
Epoch 2/15
2392/2392 [=====] - 1140s 477ms/step - loss: 0.8648 - accuracy: 0.88
Epoch 3/15
2392/2392 [=====] - 1151s 481ms/step - loss: 0.6702 - accuracy: 0.88
Epoch 4/15
2392/2392 [=====] - 1434s 599ms/step - loss: 0.5357 - accuracy: 0.90
Epoch 5/15
2392/2392 [=====] - 1414s 591ms/step - loss: 0.4321 - accuracy: 0.94
Epoch 6/15
2392/2392 [=====] - 1422s 594ms/step - loss: 0.3480 - accuracy: 0.95
Epoch 7/15
2392/2392 [=====] - 998s 417ms/step - loss: 0.2892 - accuracy: 0.953
Epoch 8/15
2392/2392 [=====] - 972s 406ms/step - loss: 0.2445 - accuracy: 0.956
```

```
Epoch 9/15
2392/2392 [=====] - 992s 415ms/step - loss: 0.2083 - accuracy: 0.960
Epoch 10/15
2392/2392 [=====] - 1004s 419ms/step - loss: 0.1829 - accuracy: 0.96
Epoch 11/15
2392/2392 [=====] - 1004s 420ms/step - loss: 0.1699 - accuracy: 0.96
Epoch 12/15
2392/2392 [=====] - 979s 409ms/step - loss: 0.1487 - accuracy: 0.966
Epoch 13/15
2392/2392 [=====] - 975s 408ms/step - loss: 0.1339 - accuracy: 0.969
Epoch 14/15
2392/2392 [=====] - 1209s 506ms/step - loss: 0.1213 - accuracy: 0.97
Epoch 15/15
2392/2392 [=====] - 1526s 638ms/step - loss: 0.1125 - accuracy: 0.97
```

Start coding or [generate](#) with AI.

```
model.save("Trained_Model.h5")
```

```
/Users/animesh/anaconda3/envs/tensorflow/lib/python3.10/site-packages/keras/src/engine/training_utils.py:110: UserWarning: saving_api.save_model(
```

```
model.save("Trained_Model.keras")
```

```
# Saving history
with open('Training_history.pkl', 'wb') as f:
    pickle.dump(training_history.history, f)
```

```
#Reloading History from pickle
with open('Training_history.pkl', 'rb') as f:
    load_history = pickle.load(f)
load_history
```

```
{'loss': [1.1453802585601807,
0.8648024201393127,
0.6702404618263245,
0.5356669425964355,
0.4320599436759949,
0.3479674160480499,
0.28923746943473816,
0.24451656639575958,
0.20830342173576355,
0.18285790085792542,
0.1699090301990509,
0.1487247496843338,
0.1339232176542282,
0.12125182151794434,
0.11251933872699738],
'accuracy': [0.8422400951385498,
0.887708306312561,
0.8857740163803101,
0.9060314893722534,
0.9422466158866882,
0.9499706029891968,
0.9534600973129272,
0.9567012786865234,
0.9605175256729126,
0.962582528591156,
0.9621120095252991,
0.9666078686714172,
0.9693262577056885,
0.9716918468475342,
0.9733254909515381],
'f1_score': [array([0.8814868 , 0.57463247, 0.00095801, 0.92311895], dtype=float32),
```

```

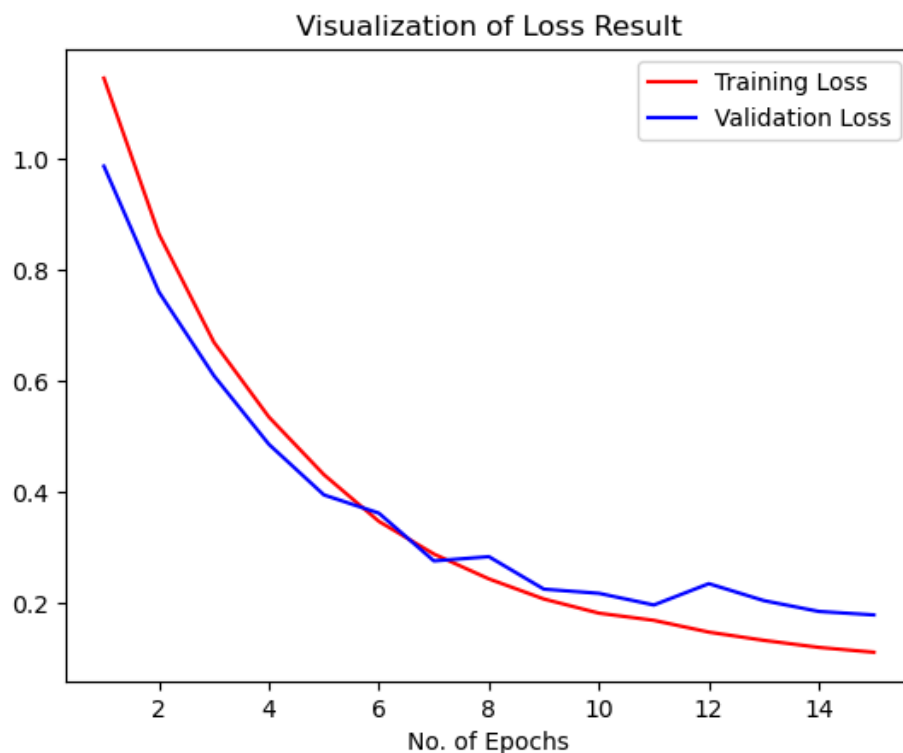
array([0.9090779 , 0.8804841 , 0.          , 0.94674116], dtype=float32),
array([0.9392844 , 0.89636165, 0.          , 0.9176859 ], dtype=float32),
array([0.9481992 , 0.90421456, 0.37415048, 0.9337706 ], dtype=float32),
array([0.95332676, 0.91403055, 0.7494094 , 0.97196394], dtype=float32),
array([0.9601239 , 0.9270588 , 0.7850574 , 0.9754878], dtype=float32),
array([0.96210384, 0.9329279 , 0.7996721 , 0.9776858 ], dtype=float32),
array([0.9665085 , 0.9369046 , 0.8150999 , 0.97792816], dtype=float32),
array([0.9699914 , 0.94476855, 0.82705045, 0.97972435], dtype=float32),
array([0.9721664 , 0.95102113, 0.83389115, 0.9800124 ], dtype=float32),
array([0.97076136, 0.9470149 , 0.83693606, 0.9804181 ], dtype=float32),
array([0.97524995, 0.95487547, 0.8535094 , 0.982151  ], dtype=float32),
array([0.97729045, 0.9588922 , 0.8669766 , 0.98323834], dtype=float32),
array([0.9785665 , 0.9606397 , 0.87928927, 0.98486435], dtype=float32),
array([0.98032624, 0.9635969 , 0.8855471 , 0.98537004], dtype=float32)],
'val_loss': [0.9870080351829529,
0.7606276869773865,
0.6100387573242188,
0.4866158068180084,
0.39565080404281616,
0.36276230216026306,
0.27686941623687744,
0.28448745608329773,
0.22604089975357056,
0.21856483817100525,
0.19756902754306793,
0.23592539131641388,
0.20539847016334534]

```

```

#Loss Visualization
epochs = [i for i in range(1,16)]
plt.plot(epochs,load_history['loss'],color='red',label='Training Loss')
plt.plot(epochs,load_history['val_loss'],color='blue',label='Validation Loss')
plt.xlabel('No. of Epochs')
plt.title('Visualization of Loss Result')
plt.legend()
plt.show()

```



```

test_set = validation_set = image_dataset_from_directory(
    'test',
    labels="inferred",
    label_mode="categorical",

```

```

class_names=None,
color_mode="rgb",
batch_size=64,
image_size=(224, 224),
shuffle=True,
seed=None,
validation_split=None,
subset=None,
interpolation="bilinear",
follow_links=False,
crop_to_aspect_ratio=False
)

```

Found 10934 files belonging to 4 classes.

```
test_loss, test_acc, f1_score = model.evaluate(test_set)
```

171/171 [=====] - 65s 368ms/step - loss: 0.1883 - accuracy: 0.9476 -

test_loss

0.18832968175411224

test_acc

0.9475946426391602

f1_score

array([0.96402097, 0.9143881, 0.78808314, 0.9702022], dtype=float32)

Start coding or [generate](#) with AI.

Model Evaluation

```
model = tf.keras.models.load_model("Trained_Model.h5")
```

```

2024-11-02 10:47:59.283337: I metal_plugin/src/device/metal_device.cc:1154] Metal device set
2024-11-02 10:47:59.283412: I metal_plugin/src/device/metal_device.cc:296] systemMemory: 16.0
2024-11-02 10:47:59.283424: I metal_plugin/src/device/metal_device.cc:313] maxCacheSize: 5.33
2024-11-02 10:47:59.283503: I tensorflow/core/common_runtime/pluggable_device/pluggable_devic
2024-11-02 10:47:59.283731: I tensorflow/core/common_runtime/pluggable_device/pluggable_devic

```

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
MobilenetV3large (Function al)	(None, 1000)	5507432
dense (Dense)	(None, 4)	4004

=====
 Total params: 5511436 (21.02 MB)
 Trainable params: 5487036 (20.93 MB)
 Non-trainable params: 24400 (95.31 KB)

```
test_set = validation_set = image_dataset_from_directory(
    'test',
    labels="inferred",
    label_mode="categorical",
    class_names=None,
    color_mode="rgb",
    batch_size=64,
    image_size=(224, 224),
    shuffle=False,
    seed=None,
    validation_split=None,
    subset=None,
    interpolation="bilinear",
    follow_links=False,
    crop_to_aspect_ratio=False
)
```

Found 10934 files belonging to 4 classes.

```
##Computing True labels from test set
true_categories = tf.concat([y for x, y in test_set], axis=0)
Y_true = tf.argmax(true_categories, axis=1)
```

Y_true

<tf.Tensor: shape=(10934,), dtype=int64, numpy=array([0, 0, 0, ..., 3, 3, 3])>

```
##Computing Predicted labels using test set
predicted_categories = model.predict(test_set)
Y_pred = tf.argmax(predicted_categories, axis=1)
```

2024-11-02 10:48:14.359978: I tensorflow/core/grappler/optimizers/custom_graph_optimizer_registry/171/171 [=====] - 31s 175ms/step

Y_true

<tf.Tensor: shape=(10934,), dtype=int64, numpy=array([0, 0, 0, ..., 3, 3, 3])>

Y_pred

<tf.Tensor: shape=(10934,), dtype=int64, numpy=array([0, 0, 0, ..., 3, 3, 3])>

✓ 1. Classification Report

```
from sklearn.metrics import classification_report
print(classification_report(Y_true,Y_pred))
```

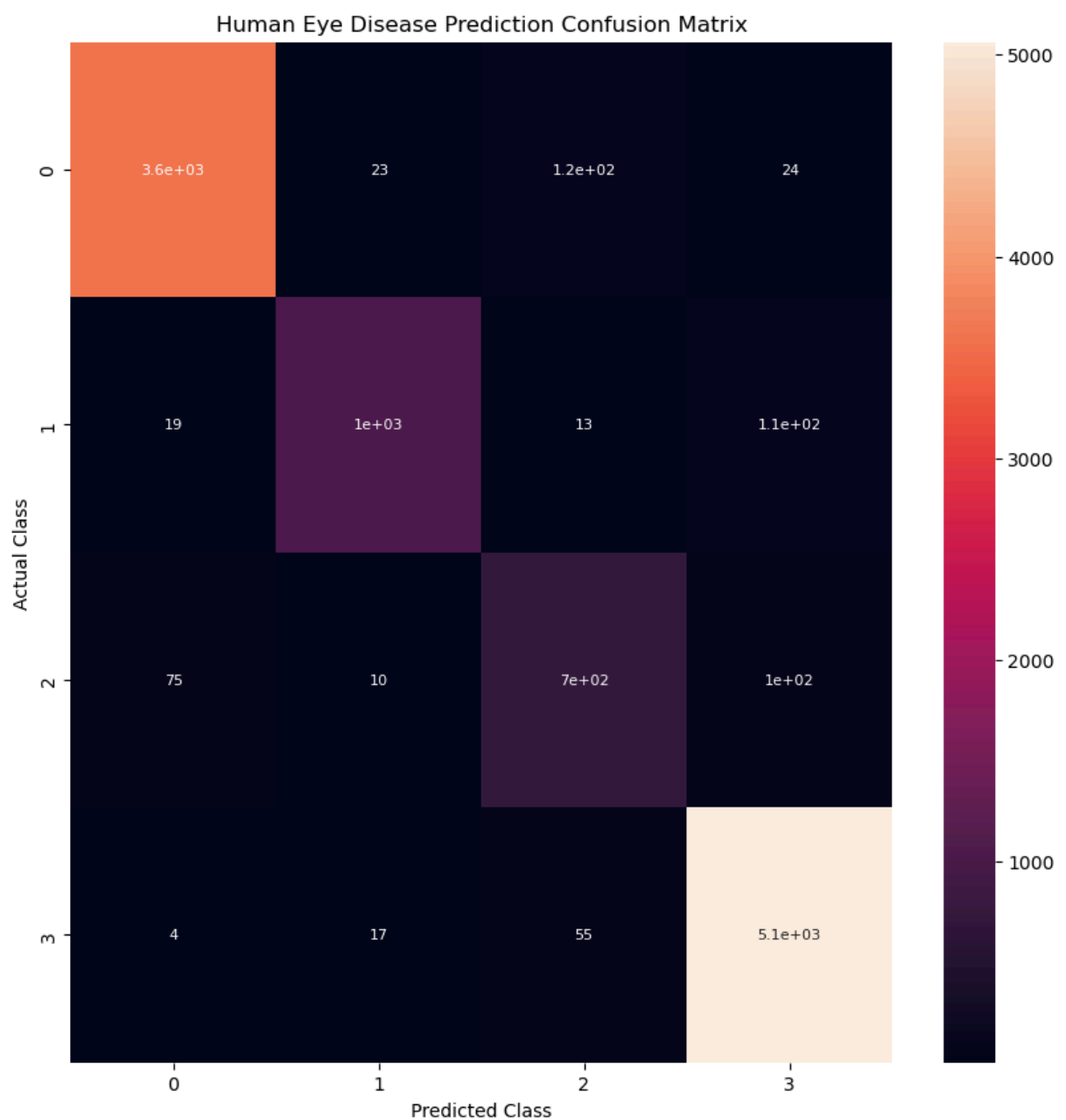
	precision	recall	f1-score	support
0	0.97	0.95	0.96	3746
1	0.95	0.88	0.91	1161
2	0.79	0.79	0.79	888
3	0.96	0.99	0.97	5139
accuracy			0.95	10934
macro avg	0.92	0.90	0.91	10934
weighted avg	0.95	0.95	0.95	10934

2. Confusion Matrix

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(Y_true,Y_pred)
cm
```

```
array([[3577, 23, 122, 24],
       [ 19, 1020, 13, 109],
       [ 75, 10, 701, 102],
       [ 4, 17, 55, 5063]])
```

```
import seaborn as sns
plt.figure(figsize=(10,10))
sns.heatmap(cm,annot=True,annot_kws={"size":8})
plt.xlabel("Predicted Class",fontsize=10)
plt.ylabel("Actual Class",fontsize=10)
plt.title("Human Eye Disease Prediction Confusion Matrix",fontsize=12)
plt.show()
```



Start coding or [generate](#) with AI.